

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00174335
Student Name:	Viktoriia Bielhrad

Module Information

Module Title:	Advanced Databases & Analytics
Module Code:	COMP H2036

Assessment Information

Assessment Title:	CA
-------------------	----

Declaration

☒ I confirm that I have read and understood the University's Academic Integrity Policy.

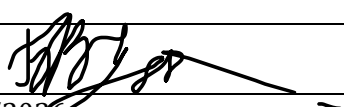
<https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/>

☒ I have completed the TU Dublin AI Driving License theory test.

<https://www.tudublin.ie/explore/about-the-university/academic-affairs/ita/ita-resources/artificial-intelligence-driving-license/>

☒ I declare that this assessment submission is my own work.

☒ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	
Date:	14/04/2026

A. Organisational Context

Small Business

B. Applied System Themes

Inventory & Supply Chain Monitoring System

Task One - Database Design (30 Marks)

- I chose to make a simple version of the **Inventory & Supply Chain Monitoring System** database for **small businesses**. When starting a business, it is important to keep everything organized: regularly keep track of the available products, purchase new ones as needed, know how much we earn and how much we must spend. All this helps the business to stay stable and profitable.

The main advantages of such a database are:

- **Efficiency Improvement**

Automation of most tasks - this greatly increases efficiency, allowing us to spend time on other business tasks instead of manually keeping records.

- **Cost Savings**

Exceeding the required amount of stock means unnecessary expenses and the need for more storage, which also costs a lot. However, if we do not replenish the stock in time, this will lead to customer dissatisfaction and loss of profit. The database covers this issue: management becomes simpler, it is clearly visible how much stock is available and how much is needed.

- **Better Customer Service**

Accordingly, if the stock is always normal, this significantly improves customer satisfaction and loyalty.

- **Data-Driven Decision Making**

When we have sales information in front of us, we can analyze certain trends and patterns that will help us better plan product purchases.

- For this database, I have defined the following **list of entities**:

- **Products**

- Primary Key – product_id
- Product_name
- Category
- Product_price
- Reorder_level

Type - strong entity

Businesses buy products from suppliers, store them and sell them - everything revolves around the existence of products in this system, so this entity is necessary here.

- **Suppliers**

- Primary Key – supplier_id
- supplier_name
- supplier_email
- supplier_phone

Type - strong entity

Businesses typically rely on suppliers to purchase raw materials or finished products for sale; without them, the business would have nothing to sell, so this entity is important in this system.

- **StockLevels**

- Primary Key – stock_id
- Quantity
- Last_updated

Type - strong entity

This table allows us to track the amount of product available, which is in short supply, and which is out of stock. This way, we can easily determine when it is time to replenish stock. This is very important for the stable operation of the business.

- **PurchaseOrders**

- Primary Key – order_id
- Order_date
- Status

Type - strong entity

This table represents the purchase of stock from suppliers. This helps to track when new stock arrives and supplier performance. This is part of supply chain monitoring.

- **PurchaseOrderItems**

- Primary Key – order_item_id
- Quantity

Type - weak entity (depends on purchaseOrders)

Shows which products were purchased from the supplier and their quantity. This is a weak entity because it cannot exist until the business places an order with the supplier.

- **Sales**
 - Primary Key – sale_id
 - Sale_date
 - Total_amount

Type - strong entity

This table shows the daily revenue that will come from sales. It is important for businesses to know this in order to understand their performance and analyze how it can be improved.

- **SalesItems**
 - Primary Key –sale_item_id
 - Quantity
 - Price_at_sale

Type - weak entity (depends on sales)

This table will store the products sold, their quantity, and the prices they were sold for. This is a weak entity because it depends on whether the sale occurs.

•

Entities	Relationship	Cardinality	Participation	Business Rule
Suppliers & products	One to many	One supplier can supply many products; Each product is supplied by one supplier.	Supplier – optional, because he may exist before products are assigned; Product – mandatory, every product needs a supplier.	The product must be linked to a supplier so that the business knows where to reorder from.
Products & stocklevels	One to one	Each product has stocklevel record; Each stocklevel refers to one product.	Product – mandatory, because we need to track the stock of each product; Stocklevels – mandatory, because stocklevel record can't exist without a product.	All products must have stocklevel records so the business can track quantity and detect when stock falls below reorder level.

Suppliers & purchaseorders	One to many	One supplier can have many orders; One order belongs to one supplier.	Supplier – optional, can exist without orders yet; Purchaseorder – mandatory, each order must be linked to supplier.	Every order must be associated with supplier because the business needs to know who is fulfilling the order.
Purchaseorders & purchaseorderitems	One to many	One order can contain many items; Each item belongs to one order.	Purchaseorders – mandatory, there is no order without items; Purchaseorderitems – mandatory, because order items cannot exist without order.	Order can include many products.
Products & purchaseorderitems	One to many	One product can appear among many orderitems; Each orderitem refers to only one product.	Product – optional, because product may exist without being ordered yet; Purchaseorderitem – mandatory, must refer to product.	Each item in a purchase order must specify which product is being ordered.
Sales & salesitems	One to many	One sale can contain many items; Each item belongs to one sale.	Sales- mandatory, a sale occurs when items are purchased; Salesitems – mandatory, cannot exist without sale.	A sale can include one or multiple products.
Products & salesitems	One to many	One product can appear among many salesitems; Each salesitem refers to one product.	Product – optional, a product may exist without being sold yet; Salesitems – mandatory, must reference a product.	Each item sold must specify which product was purchased.

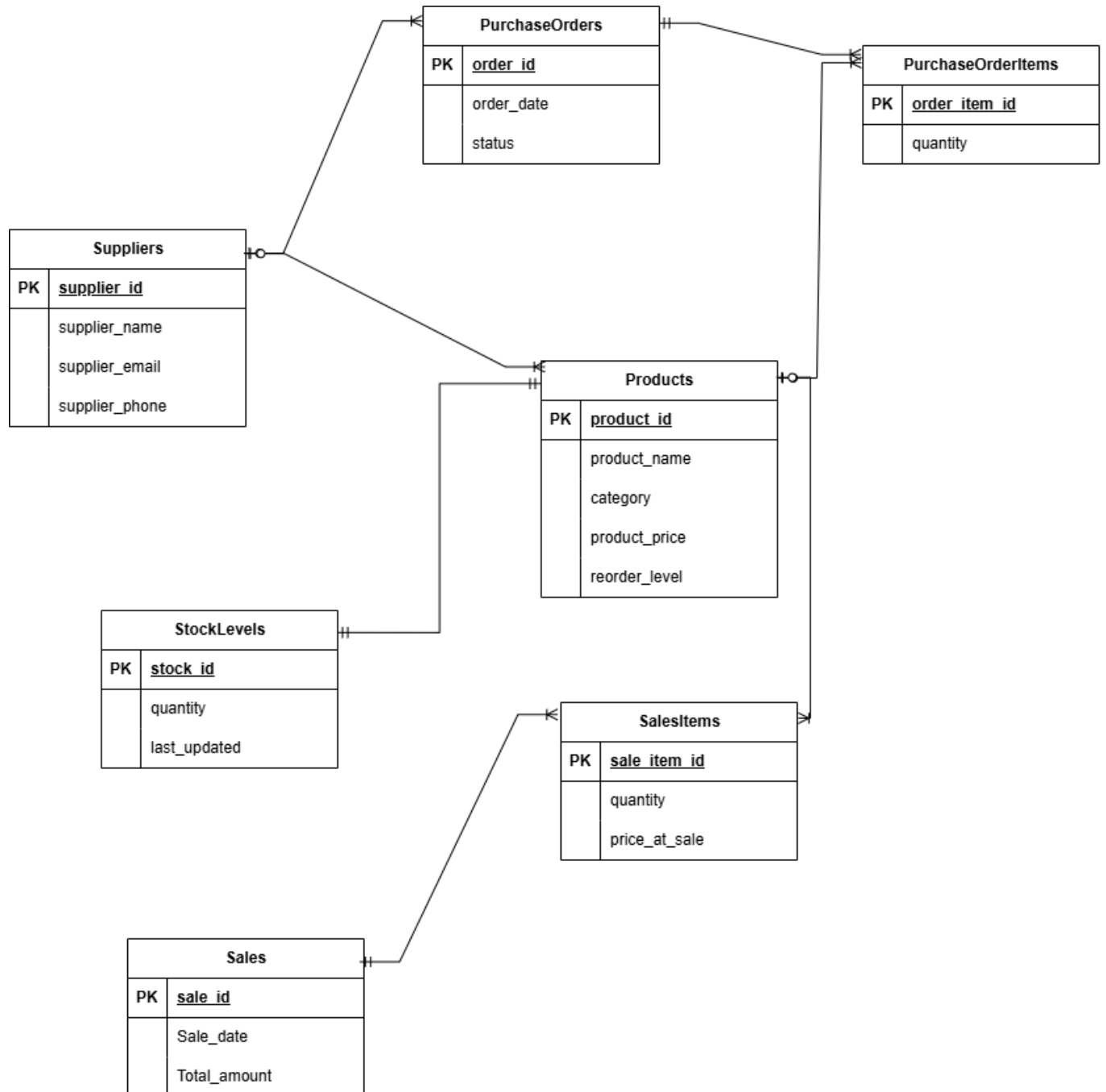


Figure 1 - initial ER diagram

xUNF

order_id	order_date	status
sale_id	sale_date	supplier_id
supplier_name	supplier_email	supplier_phone
product_id	product_name	category
product_price	reorder_level	stock_id
stock_quantity	last_updated	total-amount
sale_item_id	sale_quantity	price_at_sale
order_item_id	order_quantity	

Figure 2 - UNF

Originally everything was stored in one table. It contains repeating groups: one purchase/sale = many products/items – it makes creating product columns, querying and adding more products/items difficult. All entities are mixed in one huge table, that's why unrelated data store together, we can't make updates without affecting others, and no proper relationships can be created. Normalization can fix this.

1NF

Purchase table		
order_id	order_date	status
supplier_id	supplier_name	supplier_email
supplier_phone	product_id	product_name
category	product_price	reorder_level
order_item_id	order_quantity	

Sales table		
sale_id	sale_date	total-amount
product_id	product_name	category
product_price	price_at_sale	sale_quantity

Stock table		
product_id	product_name	stock_quantity
last_updated	stock_id	

Figure 3 - 1NF

In 1NF, each field must contain a single value, so we remove any repeating groups or multi-valued attributes so each row must represent only one product per order/ one product per sale.

2NF

Suppliers	
PK	supplier_id
	supplier_name
	supplier_email
	supplier_phone

Products	
PK	product_id
	product_name
	category
	product_price
	reorder_level
FK	supplier_id

PurchaseOrders	
PK	order_id
	order_date
	status
FK	supplier_id

PurchaseOrderItems	
PK	order_item_id
	quantity
FK	order_id
FK	product_id

Sales	
PK	sale_id
	Sale_date
	Total_amount

SalesItems	
PK	sale_item_id
	quantity
	price_at_sale
FK	sale_id
FK	product_id

StockLevels	
PK	stock_id
	quantity
	last_updated
FK	product_id

Figure 4 - 2NF

In 2NF, we need to remove partial dependencies. Because attributes like supplier details depend only on `supplier_id`, product details depend only on `product_id`, and `Total_amount` depends only on `sale_id`, etc., these must be separated into their own entities to eliminate partial dependency.

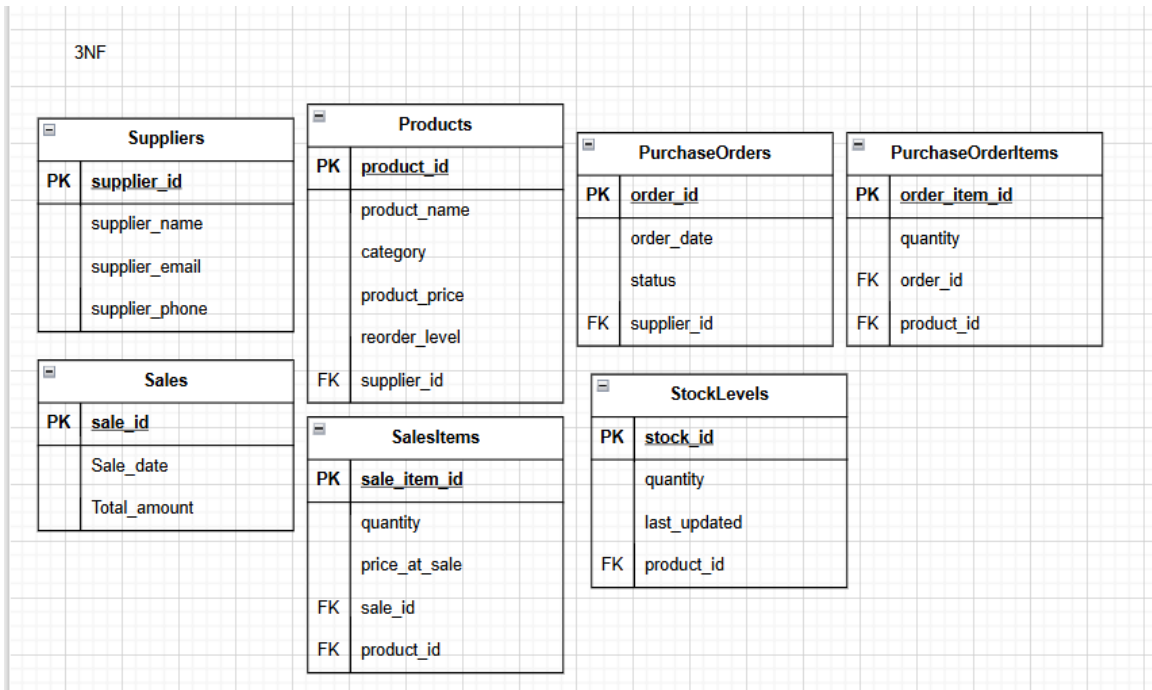


Figure 5 - 3NF

In 3NF, we remove transitive dependencies (no non-key attribute should depend on another non-key attribute). Since attributes like product details depend only on `product_id`, supplier details depend only on `supplier_id`, and stock or sales attributes depend only on their own primary keys, no transitive dependencies remain, so the tables produced in 2NF already satisfy 3NF.

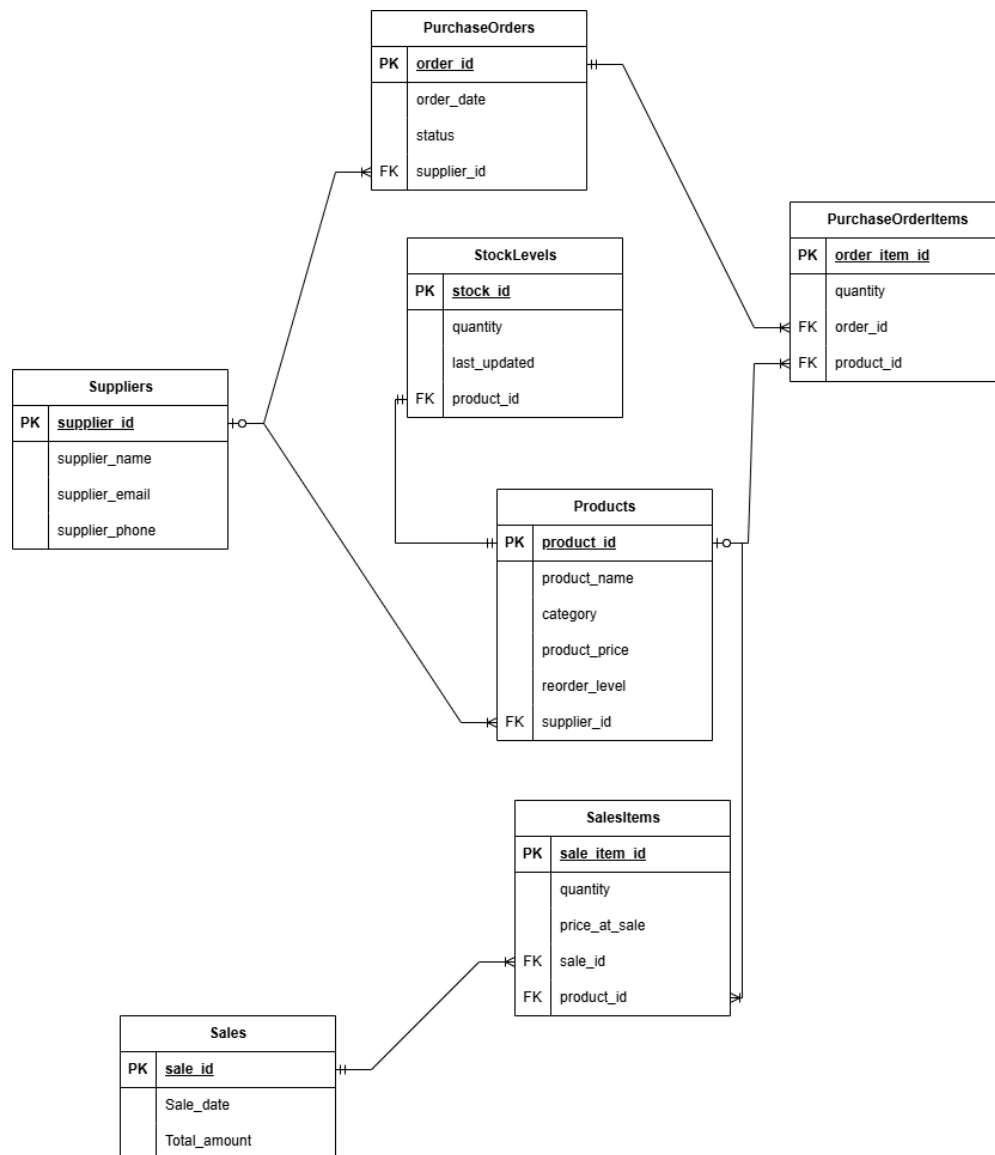


Figure 6 - final ER diagram

Task Three –Advanced SQL& Analytics

1. Q1 Non-correlated subquery

This query shows a list of products that are more expensive than the average product price.

```

SELECT
product_id,
product_name,
product_price
FROM Products
WHERE product_price > (
SELECT AVG(product_price) FROM Products
);

```

1	-- Q1 Non-correlated subquery		
2	SELECT		
3	product_id,		
4	product_name,		
5	product_price		
6	FROM Products		
7	WHERE product_price > (		
8	SELECT AVG(product_price) FROM Products		
9	);		

product_id	product_name	product_price
10	Smoked Back Bacon 20...	3.50
11	Irish Pork Sausages 40...	3.20
21	Frozen Peas 1kg	2.80
22	Frozen Chips 1.5kg	3.50
23	Frozen Pizza Margherita	3.99
24	Vanilla Ice Cream 1L	3.60
26	Basmati Rice 1kg	2.50
31	Laundry Detergent 2L	7.50
32	Kitchen Roll 4-Pack	3.80
33	Toilet Tissue 9-Pack	5.50
34	Shampoo 400ml	3.90
35	Shower Gel 400ml	3.20
36	Toothpaste 100ml	2.50
38	Granola Cereal 500g	3.40
39	Cornflakes 500g	2.60
40	Instant Coffee 200g	4.50
41	Tea Bags 80-Pack	3.20

```

1  -- Q1 Non-correlated subquery
2  EXPLAIN ANALYZE
3  SELECT
4  product_id,
5  product_name,
6  product_price
7  FROM Products
8  WHERE product_price > (
9  SELECT AVG(product_price) FROM Products
10 );

```

Data Output Messages Notifications








QUERY PLAN

text

1	Seq Scan on products (cost=12.51..25.01 rows=67 width=238) (actual time=0.208..0.214 rows=17.00 loops=1)
2	Filter: (product_price > (InitPlan 1).col1)
3	Rows Removed by Filter: 24
4	Buffers: shared hit=2
5	InitPlan 1
6	-> Aggregate (cost=12.50..12.51 rows=1 width=32) (actual time=0.037..0.037 rows=1.00 loops=1)
7	Buffers: shared hit=1
8	-> Seq Scan on products products_1 (cost=0.00..12.00 rows=200 width=16) (actual time=0.009..0.012 rows=41.00 loo...
9	Buffers: shared hit=1
10	Planning Time: 0.362 ms
11	Execution Time: 0.282 ms

2. Q2 One correlated subquery

This query shows a list of products and their total quantity ordered.

SELECT

p.product_id,

p.product_name,

(

SELECT COALESCE(SUM(poi.quantity),0)

FROM PurchaseOrderItems poi

WHERE poi.product_id=p.product_id

)AS total_quantity_ordered

FROM Products p;

```

-- Q2 One correlated subquery
SELECT
p.product_id,
p.product_name,
(
SELECT COALESCE(SUM(poi.quantity),0)
FROM PurchaseOrderItems poi
WHERE poi.product_id=p.product_id
)AS total_quantity_ordered
FROM Products p;

```

product_id [PK] integer	product_name character varying (100)	total_quantity_ordered bigint
24	Vanilla Ice Cream 1L	0
25	Spaghetti 500g	0
26	Basmati Rice 1kg	0
27	Tomato Pasta Sauce 5...	0
28	Baked Beans 415g	0
29	Tinned Chopped Tomat...	0
30	Washing-Up Liquid 500...	2334
31	Laundry Detergent 2L	0
32	Kitchen Roll 4-Pack	0
33	Toilet Tissue 9-Pack	0
34	Shampoo 400ml	0
35	Shower Gel 400ml	0

rows: 41 Query complete 00:00:00.068

At first I just used SUM in my subquery and got null. I asked copilot how to fix this and it suggested using COALESCE which would convert it to 0 which is more human readable.

```

1  -- Q2 One correlated subquery
2  EXPLAIN ANALYZE
3  SELECT
4  p.product_id,
5  p.product_name,
6  (
7  SELECT COALESCE(SUM(poi.quantity),0)
8  FROM PurchaseOrderItems poi
9  WHERE poi.product_id=p.product_id
10 ) AS total_quantity_ordered

```

Data Output Messages Notifications



QUERY PLAN	
text	
Seq Scan on products p	(cost=0.00..1014.00 rows=200 width=230) (actual time=0.018..0.420 rows=41.00 loops=1)
Buffers: shared hit=83	
SubPlan 1	
-> Aggregate	(cost=5.00..5.01 rows=1 width=8) (actual time=0.010..0.010 rows=1.00 loops=41)
Buffers: shared hit=82	
-> Seq Scan on purchaseorderitems poi	(cost=0.00..4.50 rows=200 width=4) (actual time=0.008..0.008 rows=4.88 loop=...
Filter: (product_id = p.product_id)	
Rows Removed by Filter: 195	
Buffers: shared hit=82	
Planning Time: 0.036 ms	
Execution Time: 0.428 ms	

3. Q3 One CTE (WITH) query

This query shows total revenue from product.

```

WITH product_sales AS(

SELECT

si.product_id,

SUM(si.quantity*si.price_at_sale)AS total_revenue

FROM SalesItems si

GROUP BY si.product_id

)

SELECT

ps.product_id,

```

```

p.product_name,
ps.total_revenue
FROM product_sales ps
JOIN Products p
ON p.product_id = ps.product_id;

```

```

-- Q3 One CTE (WITH) query
WITH product_sales AS(
  SELECT
    si.product_id,
    SUM(si.quantity*si.price_at_sale)AS total_revenue
  FROM SalesItems si
  GROUP BY si.product_id
)
SELECT
  ps.product_id,
  p.product_name,
  ps.total_revenue
FROM product_sales ps
JOIN Products p
ON p.product_id = ps.product_id;

```

Data Output Messages Notifications









 SQL

Showing rows: 1 to 1  Page No: 1 of 1  

product_id integer	product_name character varying (100)	total_revenue numeric
2	Semi-Skimmed Milk 1L	1306.80

```

22 -- Q3 One CTE (WITH) query
23 EXPLAIN ANALYZE
24 WITH product_sales AS(
25 SELECT

```

Data Output Messages Notifications










 SQL

QUERY PLAN

text



1	Nested Loop (cost=6.14..14.25 rows=1 width=254) (actual time=0.054..0.055 rows=1.00 loops=1)
2	Buffers: shared hit=4
3	-> HashAggregate (cost=6.00..6.01 rows=1 width=36) (actual time=0.046..0.046 rows=1.00 loops=1)
4	Group Key: si.product_id
5	Batches: 1 Memory Usage: 32kB
6	Buffers: shared hit=2
7	-> Seq Scan on salesitems si (cost=0.00..4.00 rows=200 width=15) (actual time=0.004..0.009 rows=200.00 loops=1)
8	Buffers: shared hit=2
9	-> Index Scan using products_pkey on products p (cost=0.14..8.16 rows=1 width=222) (actual time=0.008..0.008 rows=1.00 loops=1)
10	Index Cond: (product_id = si.product_id)
11	Index Searches: 1
12	Buffers: shared hit=2
13	Planning Time: 0.063 ms
14	Execution Time: 0.070 ms

4. Q4 One query using a Window Function

This query uses the window function **RANK()** to rank products within each category based on their price.

SELECT

p.product_name,

p.category,

p.product_price,

RANK()OVER(

PARTITION BY p.category

ORDER BY p.product_price

)AS price_rank

FROM Products p;

```

38  -- Q4 One query using a Window Function
39  SELECT
40  p.product_name,
41  p.category,
42  p.product_price,
43  RANK()OVER(
44  PARTITION BY p.category
45  ORDER BY p.product_price
46  )AS price_rank
47  FROM Products p;
  
```

Data Output Messages Notifications



	product_name character varying (100)	category character varying (50)	product_price numeric (10,2)	price_rank bigint
1	Blueberry Muffin	Bakery	1.50	1
2	White Sliced Pan	Bakery	1.75	2
3	Brown Sliced Pan	Bakery	1.80	3
4	Croissant 2-Pack	Bakery	2.20	4
5	Cornflakes 500g	Breakfast	2.60	1
6	Tea Bags 80-Pack	Breakfast	3.20	2
7	Granola Cereal 500g	Breakfast	3.40	3
8	Instant Coffee 200g	Breakfast	4.50	4
9	Free-Range Eggs 6-Pack	Chilled	2.30	1
10	Irish Pork Sausages 40...	Chilled	3.20	2
11	Smoked Back Bacon 20...	Chilled	3.50	3
12	Semi-Skimmed Milk 1L	Dairy	1.25	1
13	Whole Milk 1L	Dairy	1.29	2
14	Butter 250g	Dairy	2.10	3

Total rows: 41 Query complete 00:00:00.063

```

37 ON p.product_id = ps.product_id,
38 -- Q4 One query using a Window Function
39 EXPLAIN ANALYZE
40 SELECT
41   p.product_name,
42   p.category,
43   p.product_price,
44   RANK()OVER(
45     PARTITION BY p.category
46     ORDER BY p.product_price
47   )AS price_rank
  
```

Data Output Messages Notifications

QUERY PLAN

1	WindowAgg (cost=19.64..23.64 rows=200 width=360) (actual time=0.039..0.053 rows=41.00 loops=1)
2	Window: w1 AS (PARTITION BY category ORDER BY product_price ROWS UNBOUNDED PRECEDING)
3	Storage: Memory Maximum Storage: 17kB
4	Buffers: shared hit=1
5	-> Sort (cost=19.64..20.14 rows=200 width=352) (actual time=0.036..0.037 rows=41.00 loops=1)
6	Sort Key: category, product_price
7	Sort Method: quicksort Memory: 27kB
8	Buffers: shared hit=1
9	-> Seq Scan on products p (cost=0.00..12.00 rows=200 width=352) (actual time=0.004..0.006 rows=41.00 loo...
10	Buffers: shared hit=1
11	Planning Time: 0.021 ms
12	Execution Time: 0.060 ms

5. Q5 One additional Window Function query

This query calculates a running total of sales over time.

```

SELECT
  sale_date,
  total_amount,
  SUM(total_amount)OVER(
    ORDER BY sale_date
  )AS running_total
FROM Sales;
  
```

```

49  -- Q5 One additional Window Function query
50  SELECT
51  sale_date,
52  total_amount,
53  SUM(total_amount)OVER(
54  ORDER BY sale_date
55  )AS running_total
56  FROM Sales;

```

Data Output Messages Notifications



	sale_date date	total_amount numeric (10,2)	running_total numeric
1	2026-02-14	50.79	99.17
2	2026-02-14	14.16	99.17
3	2026-02-14	34.22	99.17
4	2026-02-15	40.58	139.75
5	2026-02-16	26.46	249.06
6	2026-02-16	20.31	249.06
7	2026-02-16	62.54	249.06
8	2026-02-17	59.72	361.38
9	2026-02-17	17.95	361.38
10	2026-02-17	34.65	361.38
11	2026-02-18	9.81	419.08
12	2026-02-18	47.89	419.08
13	2026-02-19	31.91	569.11
14	2026-02-19	36.54	569.11

Total rows: 200 Query complete 00:00:00.062

```

49  -- Q6 One additional window function query
50  EXPLAIN ANALYZE
51  SELECT
52  sale_date,
53  total_amount,
54  SUM(total_amount) OVER (
  
```

Data Output Messages Notifications

QUERY PLAN text

1	WindowAgg (cost=11.70..15.14 rows=200 width=42) (actual time=0.036..0.097 rows=200.00 loops=1)
2	Window: w1 AS (ORDER BY sale_date)
3	Storage: Memory Maximum Storage: 17kB
4	Buffers: shared hit=2
5	-> Sort (cost=11.64..12.14 rows=200 width=10) (actual time=0.033..0.037 rows=200.00 loops=1)
6	Sort Key: sale_date
7	Sort Method: quicksort Memory: 31kB
8	Buffers: shared hit=2
9	-> Seq Scan on sales (cost=0.00..4.00 rows=200 width=10) (actual time=0.005..0.013 rows=200.00 loo...
10	Buffers: shared hit=2
11	Planning Time: 0.018 ms
12	Execution Time: 0.106 ms

6. Q6 One query that produces a meaningful business insight

This query shows which supplier delivers the most items.

SELECT

s.supplier_name,

SUM(poi.quantity)AS total_products_delivered

FROM Suppliers s

INNER JOIN Products p

ON p.supplier_id=s.supplier_id

INNER JOIN PurchaseOrderItems poi

ON poi.product_id=p.product_id

GROUP BY s.supplier_name

LIMIT 1;

```

58  -- Q6 One query that produces a meaningful business insight
59  SELECT
60    s.supplier_name,
61    SUM(poi.quantity)AS total_products_delivered
62  FROM Suppliers s
63  INNER JOIN Products p
64  ON p.supplier_id=s.supplier_id
65  INNER JOIN PurchaseOrderItems poi
66  ON poi.product_id=p.product_id
67  GROUP BY s.supplier_name
68  LIMIT 1;
  
```

Data Output Messages Notifications










 SQL

	supplier_name character varying (100) 	total_products_delivered bigint 
1	Silverstream Household Supply	2334

	QUERY PLAN	
	text	
1	Limit (cost=24.24..24.25 rows=1 width=226) (actual time=0.111..0.112 rows=1.00 loops=1)	
2	Buffers: shared hit=5	
3	-> HashAggregate (cost=24.24..25.54 rows=130 width=226) (actual time=0.111..0.112 rows=1.00 loops=1)	
4	Group Key: s.supplier_name	
5	Batches: 1 Memory Usage: 32kB	
6	Buffers: shared hit=5	
7	-> Hash Join (cost=13.08..23.24 rows=200 width=222) (actual time=0.028..0.090 rows=200.00 loops=1)	
8	Hash Cond: (p.supplier_id = s.supplier_id)	
9	Buffers: shared hit=5	
10	-> Nested Loop (cost=0.15..9.78 rows=200 width=8) (actual time=0.014..0.056 rows=200.00 loops=1)	
11	Buffers: shared hit=4	
12	-> Seq Scan on purchaseorderitems poi (cost=0.00..4.00 rows=200 width=8) (actual time=0.005..0.012 rows=200.00 loops=1)	
13	Buffers: shared hit=2	
14	-> Memoize (cost=0.15..0.41 rows=1 width=8) (actual time=0.000..0.000 rows=1.00 loops=200)	
15	Cache Key: poi.product_id	
16	Cache Mode: logical	
17	Hits: 199 Misses: 1 Evictions: 0 Overflows: 0 Memory Usage: 1kB	
18	Buffers: shared hit=2	
19	-> Index Scan using products_pkey on products p (cost=0.14..0.40 rows=1 width=8) (actual time=0.007..0.007 rows=1.00 loops=1)	
20	Index Cond: (product_id = poi.product_id)	
21	Index Searches: 1	
22	Buffers: shared hit=2	
23	-> Hash (cost=11.30..11.30 rows=130 width=222) (actual time=0.009..0.009 rows=15.00 loops=1)	
24	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
25	Buffers: shared hit=1	
26	-> Seq Scan on suppliers s (cost=0.00..11.30 rows=130 width=222) (actual time=0.005..0.005 rows=15.00 loops=1)	
27	Buffers: shared hit=1	
28	Planning Time: 0.080 ms	
29	Execution Time: 0.135 ms	
Total rows: 29		Query complete 00:00:00.049

7. Q7 One stored procedure or function that supports business logic

This stored procedure updates the price of a product.

CREATE OR REPLACE PROCEDURE update_product_price(

```
IN p_product_id INT,  
IN p_new_price NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
UPDATE Products  
SET product_price=p_new_price  
WHERE product_id=p_product_id;  
END;  
$$;  
CALL update_product_price(1, 2.99);  
SELECT  
product_id,  
product_price  
FROM Products  
WHERE product_id=1;
```

```

70 -- Q7 One stored procedure or function that supports business logic
71 CREATE OR REPLACE PROCEDURE update_product_price(
72   IN p_product_id INT,
73   IN p_new_price NUMERIC
74 )
75 LANGUAGE plpgsql
76 AS $$
77 BEGIN
78   UPDATE Products
79   SET product_price=p_new_price
80   WHERE product_id=p_product_id;
81 END;
82 $$;
83 --CALL update_product_price(1, 2.99);
84 SELECT
85   product_id,
86   product_price
87 FROM Products
88 WHERE product_id=1;
89

```

Data Output Messages Notifications



	product_id [PK] integer	product_price numeric (10,2)
1	1	2.99

Data Output Messages Notifications



	QUERY PLAN text	
1	Index Scan using products_pkey on products (cost=0.14..8.16 rows=1 width=20) (actual time=0.002..0.002 rows=1.00 loo...	
2	Index Cond: (product_id = 1)	
3	Index Searches: 1	
4	Buffers: shared hit=2	
5	Planning Time: 0.021 ms	
6	Execution Time: 0.008 ms	

8. Q8 One trigger

This trigger logs every stock update.

```
90  -- Q8 One trigger
91  ---- create table
92  CREATE TABLE stock_history (
93  history_id SERIAL PRIMARY KEY,
94  product_id INT,
95  old_quantity INT,
96  new_quantity INT,
97  changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
98  );
99  ----create trigger function
100 CREATE OR REPLACE FUNCTION log_stock_update()
101 RETURNS TRIGGER
102 LANGUAGE plpgsql
103 AS $$
104 BEGIN
105 INSERT INTO stock_history(product_id, old_quantity, new_quantity)
106 VALUES (OLD.product_id, OLD.quantity, NEW.quantity);
107 RETURN NEW;
108 END;
109 $$;
110 ----create trigger
111 CREATE TRIGGER trigger_stock_update
112 AFTER UPDATE ON StockLevels
113 FOR EACH ROW
114 WHEN (OLD.quantity IS DISTINCT FROM NEW.quantity)
115 EXECUTE FUNCTION log_stock_update();
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 47 msec.

116

-----DEMO

117

--before update

118

SELECT *

119

FROM StockLevels

120

WHERE product_id = 1;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	stock_id [PK] integer	product_id integer	quantity integer	last_updated timestamp without time zone
1	1	1	94	2026-04-11 02:48:51.236776

```

121  --perform update
122  UPDATE StockLevels
123  SET quantity = quantity - 7
124  WHERE product_id = 1;

```

Data Output	Messages	Notifications
UPDATE 1 Query returned successfully in 45 msec.		

125

--after update

126

SELECT*

127

FROM stock_history

128

WHERE product_id = 1;

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	history_id [PK] integer	product_id integer	old_quantity integer	new_quantity integer	changed_at timestamp without time zone
1	1	1	94	87	2026-04-14 22:52:41.109633

9. Q9 Demonstrate a transaction using BEGIN / COMMIT / ROLLBACK with a realistic scenario

This transaction simulates a real sale.

When a sale occurs, a new record is entered into the Sales table (the date the sale occurred and the total amount).

Also, a record is entered into the SalesItems table about which products were sold, in what quantity, and for what price.

Then the StockLevels table is updated, because after selling 2 products, their quantity in stock should accordingly decrease by 2.

If an error occurred while running the query, a ROLLBACK would be required.

BEGIN;

INSERT INTO Sales(sale_date, total_amount)

VALUES (

CURRENT_DATE,

9.98);

INSERT INTO SalesItems(sale_id,

product_id,

quantity,

price_at_sale)

VALUES (

201,

5,

2,

4.99);

UPDATE StockLevels

SET quantity = quantity - 2

WHERE product_id = 5;

COMMIT;

--ROLLBACK;

```
129  -- Q9 Demonstrate a transaction using
130  --BEGIN / COMMIT / ROLLBACK with a realistic scenario
131  BEGIN;
132  INSERT INTO Sales(sale_date, total_amount)
133  VALUES (
134  CURRENT_DATE,
135  9.98);
136  INSERT INTO SalesItems(sale_id,
137  product_id,
138  quantity,
139  price_at_sale)
140  VALUES (
141  201,
142  5,
143  2,
144  4.99);
145  UPDATE StockLevels
146  SET quantity = quantity - 2
147  WHERE product_id = 5;
148  COMMIT;
149  --ROLLBACK;
150  |
```

Data Output Messages Notifications

COMMIT

Query returned successfully in 117 msec.

10. Q10 Briefly explain one concurrency or recovery concept relevant to your database

Multi-Version Concurrency Control (MVCC) is a technique used in databases to allow multiple users to view and update data at the same time without blocking each other.

MVCC helps prevent conflicts and reduces the need for locking, which keeps the database fast and responsive even when many users are working at the same time.

Databases often encounter concurrency problems when multiple transactions try to read or modify the same data.

If two transactions try to update the same item at the same time, MVCC can require one of them to retry the operation, ensuring consistency.

11. Q11 Demonstrate performance tuning

before

```

151 --Q11 Demonstrate performance tuning
152 ----Identify one slow analytical query and capture EXPLAIN ANALYZE before indexing
153 EXPLAIN ANALYZE
154 SELECT p.product_name,
155        SUM(si.quantity)
156 FROM Products p
157 INNER JOIN SalesItems si
158 ON si.product_id=p.product_id
159 GROUP BY p.product_name;
160

```

Data Output Messages Notifications

≡ + 📄 ▼ 📋 ▼ 🗑️ 🗄️ 📥 📈 SQL Showing rows:	
	QUERY PLAN text 🔒
1	HashAggregate (cost=10.78..12.78 rows=200 width=226) (actual time=0.084..0.084 rows=2.00 loops=1)
2	Group Key: p.product_name
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=6
5	-> Nested Loop (cost=0.15..9.78 rows=200 width=222) (actual time=0.008..0.059 rows=201.00 loops=1)
6	Buffers: shared hit=6
7	-> Seq Scan on salesitems si (cost=0.00..4.00 rows=200 width=8) (actual time=0.004..0.010 rows=201.00 loops=1)
8	Buffers: shared hit=2
9	-> Memoize (cost=0.15..0.41 rows=1 width=222) (actual time=0.000..0.000 rows=1.00 loops=201)
10	Cache Key: si.product_id
11	Cache Mode: logical
12	Hits: 199 Misses: 2 Evictions: 0 Overflows: 0 Memory Usage: 1kB
13	Buffers: shared hit=4
14	-> Index Scan using products_pkey on products p (cost=0.14..0.40 rows=1 width=222) (actual time=0.001..0.001 rows=1.00 loo...
15	Index Cond: (product_id = si.product_id)
16	Index Searches: 2
17	Buffers: shared hit=4
18	Planning Time: 0.046 ms
19	Execution Time: 0.100 ms
Total rows: 19 Query complete 00:00:00.050	

after creating index

```

160 -----Create an appropriate index and capture EXPLAIN ANALYZE after indexing
161 CREATE INDEX index_salesitems_product
162 ON SalesItems (product_id);
163 -----Compare and explain what changed
164 EXPLAIN ANALYZE
165 SELECT p.product_name,
166 SUM(si.quantity)
167 FROM Products p
168 INNER JOIN SalesItems si
169 ON si.product_id=p.product_id
170 GROUP BY p.product_name;
  
```

Data Output Messages Notifications

Showing rows: 1

	QUERY PLAN
	text
1	HashAggregate (cost=10.82..12.82 rows=200 width=226) (actual time=0.156..0.157 rows=2.00 loops=1)
2	Group Key: p.product_name
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=6
5	-> Nested Loop (cost=0.15..9.81 rows=201 width=222) (actual time=0.014..0.123 rows=201.00 loops=1)
6	Buffers: shared hit=6
7	-> Seq Scan on salesitems si (cost=0.00..4.01 rows=201 width=8) (actual time=0.007..0.017 rows=201.00 loops=1)
8	Buffers: shared hit=2
9	-> Memoize (cost=0.15..0.41 rows=1 width=222) (actual time=0.000..0.000 rows=1.00 loops=201)
10	Cache Key: si.product_id
11	Cache Mode: logical
12	Hits: 199 Misses: 2 Evictions: 0 Overflows: 0 Memory Usage: 1kB
13	Buffers: shared hit=4
14	-> Index Scan using products_pkey on products p (cost=0.14..0.40 rows=1 width=222) (actual time=0.003..0.003 rows=1.00 loo...
15	Index Cond: (product_id = si.product_id)
16	Index Searches: 2
17	Buffers: shared hit=4
18	Planning Time: 0.083 ms
19	Execution Time: 0.174 ms

The cost changed only slightly. Scan type remained the same.

Both before and after indexing the plan is almost identical. This means tables are small, query must read all rows anyway and join already used the pk index.

After creating the index, the execution plan remained almost unchanged, because the table was small and the query still had to read all the rows to find the sum, so the execution time would not be as fast as shown in the screenshots(0.100ms->0.174ms).

The main difference is that postgresql now has an index available for future queries.

Although the execution time did not decrease, the plan shows that the database can now use the index for searches, which will improve performance on larger tables(thousands of rows).

12. Q12 Create at least two roles

I created two postgres roles: admin and staff. The admin role has full permissions on all tables, while the staff role has read-only access on few tables. I then created two users and assigned the appropriate roles to them.

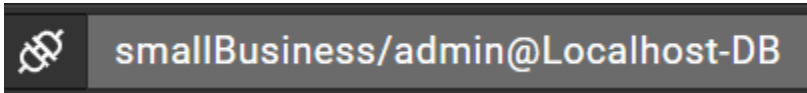
```
171 -- Q12 Create at least two roles
172 ----Admin
173 --create role
174 CREATE ROLE admin;
175 --grant permissions
176 GRANT SELECT, INSERT, UPDATE, DELETE ON Products TO admin;
177 GRANT SELECT, INSERT, UPDATE, DELETE ON Sales TO admin;
178 GRANT SELECT, INSERT, UPDATE, DELETE ON SalesItems TO admin;
179 GRANT SELECT, INSERT, UPDATE, DELETE ON StockLevels TO admin;
180 GRANT SELECT, INSERT, UPDATE, DELETE ON stock_history TO admin;
181 GRANT SELECT, INSERT, UPDATE, DELETE ON Suppliers TO admin;
182 GRANT SELECT, INSERT, UPDATE, DELETE ON PurchaseOrders TO admin;
183 GRANT SELECT, INSERT, UPDATE, DELETE ON PurchaseOrderItems TO admin;
184 --create user
185 CREATE USER user_admin WITH PASSWORD 'pass1';
186 --assign role
187 GRANT admin TO user_admin;
188 ----Staff
189 --create role
190 CREATE ROLE staff;
191 --grant permissions
192 GRANT SELECT ON Products TO staff;
193 GRANT SELECT ON Sales TO staff;
194 GRANT SELECT ON SalesItems TO staff;
195 GRANT SELECT ON StockLevels TO staff;
196 GRANT SELECT ON stock_history TO staff;
197 --create user
198 CREATE USER user_staff WITH PASSWORD 'pass2';
199 --assign role
200 GRANT staff TO user_staff;
201
```

Data Output Messages Notifications

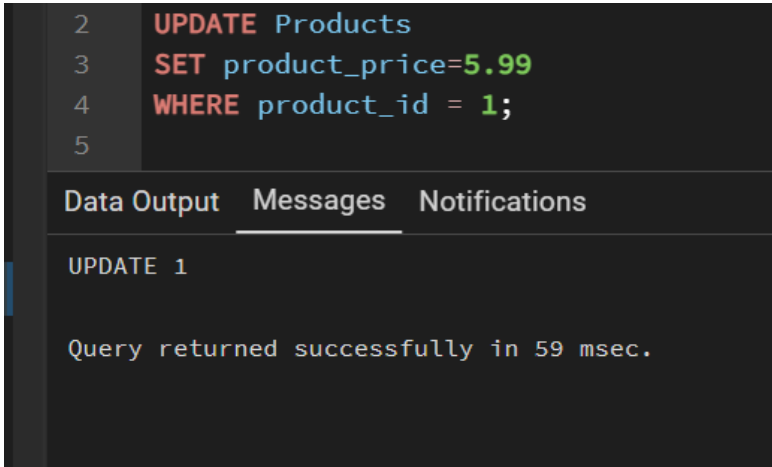
GRANT ROLE

Query returned successfully in 44 msec.

So, I logged in as admin:

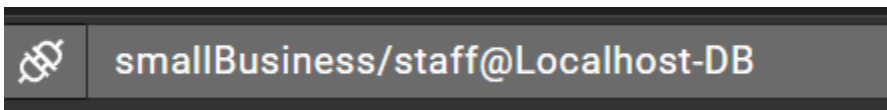


Next I try to run the UPDATE:

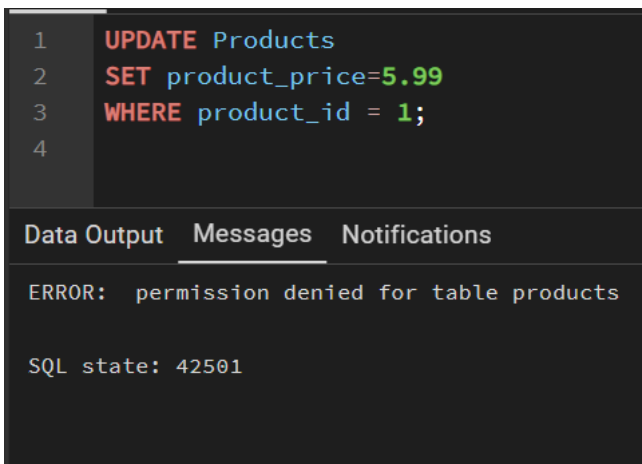


Query executed successfully because as admin I have a permission to update.

Now I log in as a staff member:



Then I try to run the same update query:



Obviously I'm getting an error, because staff doesn't have permission to update. That's because I haven't granted that permission.

```

1 SELECT *
2 FROM Products;

```

Data Output Messages Notifications						
Showing rows: 1 to 41 Page No: 1						
	product_id [PK] integer	product_name character varying (100)	category character varying (50)	product_price numeric (10,2)	reorder_level integer	supplier_id integer
1	2	Semi-Skimmed Milk 1L	Dairy	1.25	10	3
2	3	Cheddar Cheese 200g	Dairy	2.49	8	3
3	4	Butter 250g	Dairy	2.10	8	3
4	5	Brown Sliced Pan	Bakery	1.80	12	5
5	6	White Sliced Pan	Bakery	1.75	12	5
6	7	Croissant 2-Pack	Bakery	2.20	6	5
7	8	Blueberry Muffin	Bakery	1.50	6	5
Total rows: 41		Query complete 00:00:00.110				

However, I can perform the select. This is because I granted this permission to the staff user.

Using AI

In general, I used AI for the following tasks:

1. Brainstorming - to come up with some attributes, advise which tables a small business should have in its database besides the ones I came up with, ideas and tips for queries
2. Validation - sometimes there were errors and I didn't know how to fix them - mostly typos that I didn't notice
3. Generating table rows (insert statements)
4. Explanations

References:

1. Secure_Excels_User_. (2024, June 17). *Why your business needs an inventory Database: Key benefits explained*. ExcelHunters. <https://excelhunters.com/blog-post/why-your-business-needs-an-inventory-database-key-benefits-explained/>
2. GeeksforGeeks. (2025, December 6). *What is MultiVersion Concurrency Control (MVCC) in DBMS?* GeeksforGeeks. <https://www.geeksforgeeks.org/dbms/what-is-multi-version-concurrency-control-mvcc-in-dbms/>